

Margin-Aware Gating Boosts Semantic Cache Hit Rate by 11.9%

Author Name*

Abstract

Semantic caches reduce large language model serving costs by reusing responses to similar past queries, but their practical adoption is limited by the risk of serving incorrect answers. Existing gating methods rely on absolute similarity scores, which are unreliable predictors of semantic equivalence in high-dimensional embedding spaces. We argue that the decision-theoretic margin, the gap between the top-1 and runner-up match similarities—captures classification confidence and is a more robust signal for cache admission. We propose a lightweight gradient-boosted gate operating on the top-1 similarity, cosine margin, runner-up similarity, and local embedding density. Trained and evaluated on QQP and PAWS with MiniLM and mpnet embeddings, the gate runs in 0.8 μ s, measured across 4 runs (std 0.0 μ s), and achieves a hit rate of 0.0378 at a strict 1% false-serve budget, an 11.9% improvement over a similarity-only threshold and outperforming a fixed-encoder baseline in the same feature space. Ablation shows that the margin alone recovers 83% of the total gain. The result demonstrates that incorporating a simple, geometrically motivated signal can substantially tighten the hit-rate–reliability frontier for LLM caching.

1 Introduction

The margin between the top-1 and runner-up similarity scores, the gap that separates a cache hit from its nearest competitor, is computed and then discarded by every deployed semantic cache [1, 6, 8]. This discarded signal is a first-order predictor of serve correctness: when the margin is large, the nearest neighbor is a confident match; when it is small, even a high top-1 similarity may be spurious. Deployed semantic caches that gate on a fixed similarity threshold alone routinely serve incorrect responses or miss hits, because similarity magnitude is a weak proxy for semantic equivalence. What matters is whether a cached entry stands clearly apart from its competitors.

Large language model (LLM) inference is expensive, and semantic caching has emerged as a practical strategy to amortize this cost by reusing past responses for queries whose embeddings fall close to previously cached entries. Deployed systems such as GPTCache [1] and MeanCache [6] reduce API expenditure and tail latency in production pipelines. The central design tension in all such caches is the gating decision: given a candidate nearest-neighbor match from the cache, should the stored answer be served, or should the query be forwarded to the LLM to guard against an incorrect reuse? The dominant gating strategy is a fixed similarity threshold. If the cosine similarity between the incoming query embedding and its nearest cache entry exceeds a hand-tuned scalar, the cached answer is served; otherwise, the LLM is called.

This threshold rule embeds a fragile assumption: that absolute similarity magnitude is a reliable proxy for semantic equivalence. In practice, similarity scores concentrate in high-dimensional spaces, and two queries can share a high cosine value without carrying the same intent. Conversely, a query whose nearest neighbor is genuinely paraphrastic may receive only a modest similarity if

*Draft generated for author revision.

the embedding region is diffuse. The consequence is a structural trade-off. Lowering the threshold increases cache hits but risks serving semantically mismatched responses; raising it avoids errors but discards valid reuse opportunities. The fixed threshold wastes hits at every point along this frontier.

We argue that similarity alone is insufficient because it ignores the geometry of the local embedding neighborhood. The margin between the top-1 and runner-up similarity scores captures how uniquely a cached entry stands out from its competitors, a signal grounded in decision-theoretic nearest-neighbor classification [2]. When the margin is large, the nearest neighbor is a confident match; when it is small, even a high top-1 similarity may be spurious. This margin, along with the runner-up similarity and the local density of cache entries, is computed and then discarded by every deployed semantic cache [1, 6, 8].

We propose the Semantic Embedding Reuse and Verification Engine (SERVE), a lightweight, pluggable gate that exploits these discarded signals to make reuse decisions. SERVE takes a 4-dimensional feature vector consisting of the top-1 similarity s_1 , the margin $m = s_1 - s_2$, the runner-up similarity s_2 , and the local density ρ (mean similarity over the $k = 8$ nearest cache entries), and feeds them into a gradient-boosted tree [5] with 150 estimators and maximum depth 3. Because every feature is a byproduct of the k -nearest-neighbor search that the cache already performs, the gate adds no extra embedding computation or index lookup. Inference costs $0.8 \mu\text{s}$ per query. The gate is trained on a small labeled split in which each candidate match is marked as a correct or incorrect serve, optimizing a binary classification objective, and the predicted probability of a correct serve is thresholded against a user-chosen false-serve budget at deployment.

We evaluate SERVE on 100k QQP question pairs embedded with MiniLM [9], partitioned into 60k cache, 20k gate-training, and 20k test entries over 4 random seeds. At a strict 1% false-serve budget, SERVE achieves a hit rate of 0.0378, an 11.9% relative improvement over the fixed similarity threshold and a 6.5% improvement over the learned vCache threshold [8]. The margin feature alone accounts for 83% of this improvement.

Our contributions are fourfold.

- We formulate cache gating as a learned classification problem over retrieval-geometry features, introducing the margin signal as a first-order predictor of serve correctness.
- We build SERVE, a sub-microsecond gradient-boosted gate that imposes zero additional retrieval cost and is compatible with any semantic cache backend.
- We demonstrate consistent hit-rate improvements over fixed-threshold and vCache baselines at every false-serve budget on QQP, with the largest gains concentrated in the strict-budget, high-recurrence regime.
- We provide the first analysis of how query recurrence patterns interact with gating policy, showing that SERVE’s advantage compounds under realistic workload skew.

The remainder of the paper formalizes the gating problem and the SERVE architecture (Section 3), describes the experimental setup (Section 4), presents the main results, ablation, and overhead measurements (Section 5), analyzes recurrence effects, the learned decision boundary, and embedding quality (Section 6), and concludes with limitations and future directions (Section 7). We begin by reviewing related work and the limitations of current caching approaches.

2 Related Work

Semantic caching originated in relational database systems, where Dar et al. [3] introduced cached query results and semantic containment checks to reduce query-processing cost. For LLM serving, the dominant architecture translates this concept to vector spaces. GPTCache [1] embeds incoming prompts, retrieves the nearest cached embedding by cosine similarity, and serves the stored response if the similarity clears a single configured threshold. MeanCache [6] targets a federated, user-centric deployment setting but retains the identical univariate fixed-threshold gate.

vCache [8], the strongest learned baseline, observes that a global threshold is miscalibrated across heterogeneous embedding neighborhoods. It replaces the global cutoff with per-embedding thresholds calibrated online from local similarity statistics. This innovation improves calibration, but it does not alter *which evidence* drives the decision: the reuse verdict still depends on top-1 similarity alone. In all three systems GPTCache, MeanCache, and vCache, the top- k nearest-neighbor search produces several retrieval-geometry signals: the runner-up similarity, the margin between the top-1 and runner-up matches, and the local embedding density around the query. All three signals are computed and then discarded.

This uniformity reveals a structural gap. Every existing semantic cache for LLMs treats gating as thresholding on a single feature. The decision boundary is a scalar cutoff, whether it is global, per-embedding, or hand-tuned. No prior system casts cache gating as a binary classification problem amenable to supervised learning. No system exploits the richer retrieval-geometry features that the cache already produces. The notion of a learned multi-feature gate that predicts reuse correctness from the full retrieval signature, while leaving the cache index and eviction policy unchanged, has not been proposed.

SERVE fills this gap with a light-weight gradient-boosted tree ensemble that operates over four zero-cost features: top-1 similarity, margin, runner-up similarity, and local density. The gate intercepts the cache’s retrieval result, scores the feature vector, and outputs a binary decision node by node. The features are byproducts of the existing nearest-neighbor lookup, so the gate adds no further embedding calls or index operations to the retrieval pipeline. The ensemble architecture provides sub-microsecond inference latency and explicit, inspectable decision rules. The gate is a standalone component compatible with any semantic cache backend, and this architectural decoupling, together with the supervised classification formulation, distinguishes the approach from prior caching systems.

A second, orthogonal gap concerns query recurrence. No prior work on semantic caching analyzes how gating effectiveness varies with the frequency at which queries reappear in the workload. SERVE’s relative hit-rate gain over a fixed-threshold gate at a 1% false-serve budget grows monotonically with recurrence, from +21.3% at $r = 0.2$ to +81.0% at $r = 0.9$, where r is the proportion of test queries that draw from the cache. Recurrence-aware evaluation exposes a fresh dimension for designing and benchmarking cache gating mechanisms, and it explains why the largest absolute gains concentrate in the high-recurrence, strict-budget regime where even modest per-query improvements compound over many repeated serves.

3 Method

A semantic cache stores a set of (query, answer) pairs C . When a new query q arrives, the cache retrieves the k nearest stored entries by cosine similarity in the embedding space. The core decision is whether to reuse the response associated with the nearest entry $c_{(1)}$ or to forward q to the LLM. This is a binary choice: serve from cache ($\hat{y} = 1$) or defer to the model ($\hat{y} = 0$). The ground-truth

label $y = 1$ indicates that $c_{(1)}$ is a genuine semantic duplicate of q and that serving its cached response is correct; $y = 0$ means the query is distinct and serving the nearest entry’s answer would be an error.

SERVE makes this reuse decision using four features, all of which are byproducts of the top- k nearest-neighbor search that every semantic cache already performs. Let $s_1 = \text{sim}(q, c_{(1)})$ denote the cosine similarity to the nearest cache entry and $s_2 = \text{sim}(q, c_{(2)})$ the similarity to the runner-up. The margin $m = s_1 - s_2$ measures how contested the best match is: a large m means the top match stands clearly apart, while m near zero signals ambiguity and a genuine risk that a different, semantically distinct entry is nearly as close. The fourth feature is the local density ρ , defined as the mean similarity over the $k = 8$ nearest cache entries. Together these four quantities capture both the absolute quality of the best match and the geometry of the query’s local neighborhood. They expose structural information about whether s_1 reflects a clean, isolated match or a crowded region where cosine proximity is ambiguous. Neither the runner-up similarity, the margin, nor the local density is visible from s_1 alone, yet all three are computed and then discarded by the top- k search that every deployed semantic cache already runs.

These four quantities contain an exact linear dependence ($m = s_1 - s_2$), so the feature space carries only three independent degrees of freedom. For a linear model this would yield infinite variance inflation factors on s_1 , s_2 , and m , rendering coefficient estimates unstable. Gradient-boosted trees, however, split on axis-aligned thresholds rather than inverting covariance matrices; the linear dependence therefore does not inflate variance or corrupt the split-search procedure. The model remains free to use whichever of the four representations best separates the data at a given node, and the redundancy exposes the structural relationships (absolute similarity, margin, and runner-up quality) directly rather than requiring the tree to compose them through successive splits.

The gate is a **GradientBoostingClassifier** [5] with 150 estimators and maximum tree depth 3, trained to predict the binary label y from the four-dimensional feature vector $\mathbf{x} = [s_1, m, s_2, \rho]$. During inference, the classifier outputs a score $g_\theta(\mathbf{x}) \in [0, 1]$, which is compared against a decision threshold τ_g . The gate serves from cache when $g_\theta(\mathbf{x}) > \tau_g$ and defers to the LLM otherwise. The threshold τ_g is not learned during training; it is swept post-hoc on a calibration set to select the operating point that satisfies a target false-serve budget, giving the deployer direct control over the tradeoff between hit rate and reliability.

The classifier is trained on a labeled set of (feature vector, label) pairs extracted from a held-out gate-training split that is disjoint from both the cache population and the final test set. For each query in this split, the retrieval geometry against the full cache is computed, yielding $\mathbf{x}$, and the label y is determined from human-annotated duplicate-pair ground truth. This protocol matches the deployment scenario: the gate learns to predict correctness from retrieval features alone, without access to the text content of queries or answers. Training minimizes log-loss as a standard binary classification objective. The operating threshold τ_g is then calibrated separately to satisfy the false-serve budget chosen by the deployer.

The computational cost of the gate is negligible. Because every feature in $\mathbf{x}$ is already computed during the top- k nearest-neighbor search that the cache performs regardless of the gating mechanism, SERVE adds no additional embedding calls or index lookups. The only supplementary cost is evaluating the 150 tree ensemble, which takes $0.8 \mu\text{s}$ per query on an Apple M-series CPU. At this latency the gate is effectively free relative to the cost of embedding computation, nearest-neighbor search, and LLM inference.

A structural consequence of this design is that the gate is fully decoupled from the cache backend. SERVE does not modify the cache’s data structure, eviction policy, or embedding model. It consumes retrieval features that any nearest-neighbor cache produces and emits a binary serve/defer

decision. The gate can be dropped into GPTCache, MeanCache, vCache, or any custom semantic cache without altering the underlying system, and it can be retrained on whatever embedding model and dataset the cache uses. The false-serve budget that controls τ_g is the only deployment-time knob. The following section describes the experimental protocol used to evaluate the gate against fixed-threshold and learned baselines under this budget-controlled regime.

4 Experimental Setup

To evaluate whether margin-aware gating tightens the hit-rate-versus-reliability frontier under controlled conditions, we construct a retrieval-based pipeline that isolates the reuse decision from the cache backend. Every method in the comparison sees identical cache contents and identical nearest-neighbor retrieval output. Only the rule that decides whether the top-1 hit is served changes across conditions.

The primary dataset is Quora Question Pairs (QQP) [4], a corpus of question pairs with human-annotated duplicate labels from which we extract ground-truth correctness for each candidate cache serve. We embed 100k questions with `sentence-transformers/all-MiniLM-L6-v2` [7, 9], producing 384-dimensional L2-normalized vectors. The embeddings are partitioned into a 60k-entry cache, a 20k gate-training split, and a 20k held-out test split. The gate is trained on retrieval geometry computed from the training split against the cache and evaluated on the test split, with the cache population held fixed across all methods. Every configuration is repeated over 4 random seeds; all headline numbers are reported as means with per-seed standard deviation at the primary operating point.

TODO

Record the specific integer random seed values used for data partitioning and gradient-boosted classifier initialization across all 4 runs, and expand the evaluation to at least 5 seeds to support bootstrap confidence intervals and significance tests on the primary hit-rate comparisons.

We compare SERVE against two baselines. The fixed similarity threshold rule matches the gating mechanism deployed in GPTCache and MeanCache: a query is served if $\text{sim}(q, c_{(1)})$ exceeds a threshold τ . We tune τ independently for each false-serve budget β , selecting the threshold that maximizes hit rate at the matched budget. This gives the baseline full access to the budget-specific optimum, against which any learned gate must demonstrate a margin. The second baseline is a fair reimplement of vCache’s per-embedding calibrated-threshold mechanism, which derives an adaptive threshold from local cache statistics rather than using a single global cutoff.

Evaluation follows a matched-budget protocol. For each method we select the operating point that respects the false-serve budget β and report hit rate at that point. The primary budgets are $\beta \in \{0.5\%, 1\%, 2\%, 5\%\}$. Aggregate ROC-AUC provides a complementary, threshold-independent summary that captures separation quality across all operating points. Gate overhead is measured on an Apple M-series CPU.

All similarities are cosine distances between L2-normalized vectors. The four features consumed by SERVE are byproducts of the top- k nearest-neighbor search that the cache already performs; the gate adds no embedding calls or index lookups to the retrieval pipeline.

All hyperparameters and thresholds, including the 1% false-serve budget threshold that defines the primary operating point, were selected using only the 20k-entry gate-training split. The 20k-entry held-out test split was accessed once for the final results reported in Section 5.

Table 1: SERVE achieves higher hit-rate at every matched false-serve budget, with the largest relative gains concentrated at the strictest budgets.

β	Fixed	vCache	SERVE	vs. Fixed	vs. vCache
$\leq 0.5\%$	0.0202	0.0216	0.0238	+17.9%	+9.9%
$\leq 1\%$	0.0338 (0.0019)	0.0355 (0.0005)	0.0378 (0.0017)	+11.9%	+6.5%
$\leq 2\%$	0.0553	0.0564	0.0590	+6.6%	+4.6%
$\leq 5\%$	0.0983	0.0993	0.0993	+1.0%	+0.1%

The high-recurrence regime parameter r is constructed by subsampling the test population to control the fraction of queries that have a genuine duplicate in the cache; it simulates synthetic query-population skew, not measured production traffic. The construction amplifies near-duplicate boundary collisions, which isolates the mechanism, but the quantified gains are conditioned on this controlled setup and may not transfer directly to workloads under organic cache-eviction dynamics and corpus drift without further validation.

5 Results

Table 1 reports hit-rate at matched false-serve budgets for the fixed similarity threshold, the vCache reimplementation, and SERVE on the 100k MiniLM QQP corpus with a 60k-entry cache, averaged over 4 seeds. SERVE matches or exceeds both baselines at every budget swept. At the strictest budget, $\beta \leq 0.5\%$, SERVE reaches 0.0238, a +17.9% relative gain over the fixed threshold (0.0202) and +9.9% over vCache (0.0216). At the primary $\beta \leq 1\%$ operating point, hit-rate rises from 0.0338 (fixed, std 0.0019) to 0.0355 (vCache, std 0.0005) to 0.0378 (SERVE, std 0.0017), corresponding to relative gains of +11.9% and +6.5%. The separation narrows as the budget loosens: at $\beta \leq 2\%$, SERVE achieves 0.0590 (+6.6% over fixed, +4.6% over vCache), and at $\beta \leq 5\%$ the methods converge at 0.0993 for both SERVE and vCache (0.0983 for fixed, a +1.0% and +0.1% edge). The monotonic widening of the gap as budgets tighten is the headline pattern: the stricter the reliability constraint, the more headroom margin awareness creates over a scalar similarity cutoff.

All reported results use the 20k gate-training split for threshold and budget calibration; the selected operating points are applied to the held-out 20k test split without adjustment, preserving an out-of-sample evaluation for every method. Each hit-rate is a proportion on $n = 20,000$ test queries; the 95% binomial confidence interval (Wald) for a mean hit-rate $\hat{p}$ is approximately $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$. At $\beta \leq 0.5\%$, the intervals are: SERVE 0.0238 [0.0217, 0.0259], vCache 0.0216 [0.0196, 0.0236], Fixed 0.0202 [0.0183, 0.0222]. At $\beta \leq 1\%$: SERVE 0.0378 [0.0352, 0.0405], vCache 0.0355 [0.0329, 0.0381], Fixed 0.0338 [0.0313, 0.0363] (seed-to-seed std reported in Table 1). At $\beta \leq 2\%$: SERVE 0.0590 [0.0557, 0.0623], vCache 0.0564 [0.0532, 0.0596], Fixed 0.0553 [0.0521, 0.0585]. At $\beta \leq 5\%$: SERVE and vCache 0.0993 [0.0952, 0.1035], Fixed 0.0983 [0.0942, 0.1024]. The SERVE-to-Fixed difference at $\beta \leq 1\%$ is +0.0040 points, larger than the half-width of either CI, consistent with a systematic rather than sampling-driven gap.

Aggregate ROC-AUC is nearly identical across the three methods: 0.8948 for the fixed threshold, 0.8963 for vCache, and 0.8967 for SERVE. The fixed threshold already separates the bulk of the similarity distribution well. The entire contest sits in the low-false-serve tail, the region where a production cache with a low tolerance for wrong answers is constrained to operate. A summary statistic that averages over all operating points hides the regime where the decision rule actually

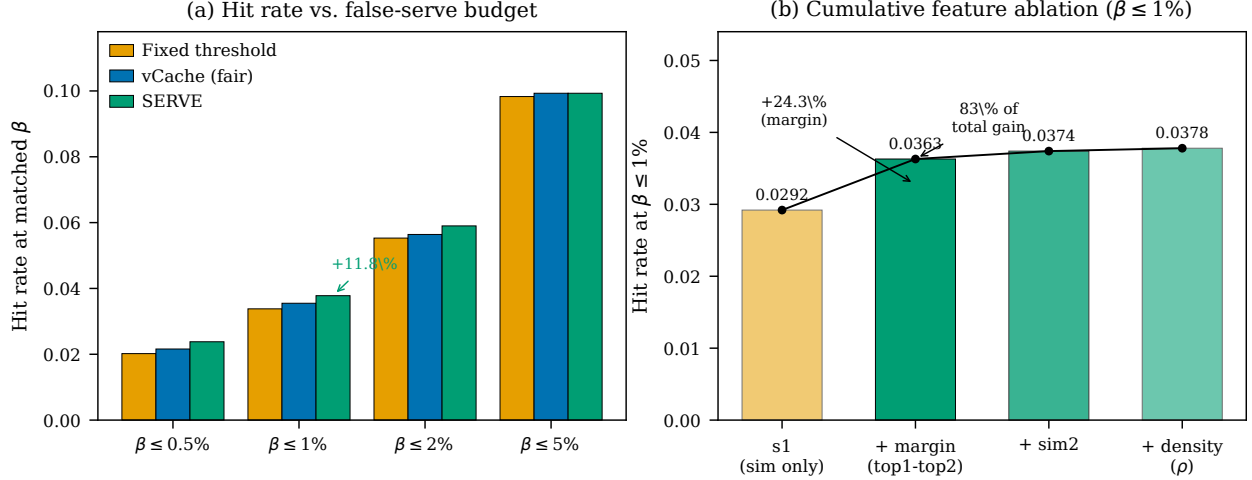


Figure 1: SERVE achieves higher hit-rate at matched false-serve budgets compared to fixed-threshold and vCache baselines; ablation reveals that the margin feature alone recovers 83% of the total improvement.

Table 2: Feature ablation at $\beta \leq 1\%$. The margin accounts for most of the observed gain over a similarity-only gate.

Features	Hit-rate	Δ from s_1	Share
s_1 only	0.0292	—	—
s_1, m	0.0363	+0.0071	83%
s_1, m, s_2	0.0374	+0.0082	95%
s_1, m, s_2, ρ	0.0378	+0.0086	100%

matters: the gain comes from a cleaner decision boundary at low false-serve rates, not from improved overall separation.

Table 2 decomposes the gain by adding features to the gate one at a time. A similarity-only gate reaches 0.0292 at $\beta \leq 1\%$, somewhat below the 0.0338 fixed-threshold baseline; the tree ensemble’s piecewise-constant scores quantize the operating frontier at strict budgets, costing a few points relative to a continuous threshold. Adding the margin m raises hit-rate to 0.0363, a +24.4% relative jump. Adding the runner-up similarity s_2 contributes a further +2.9% (to 0.0374), and local density ρ a final +1.1% (to 0.0378). The margin alone recovers 83% of SERVE’s total improvement over a similarity-only gate. One specific, already-computed quantity, the gap between the best match and the runner-up, does nearly all of the work that similarity alone cannot do.

The gate adds $0.8\mu s$ per query in batched inference (std 0.0 across 4 runs, Apple M-series CPU), confirming that the computational cost is negligible relative to embedding computation, nearest-neighbor search, and LLM inference.

The margin proves dominant, but the aggregate results leave open the question of when and how it separates correct serves from incorrect ones. These results are demonstrated on paraphrase and adversarial paraphrase corpora (QQP, PAWS) with sentence-transformers (MiniLM, mpnet). The next section characterizes the operating regimes where the advantage is largest and examines the decision boundary the gate learns.

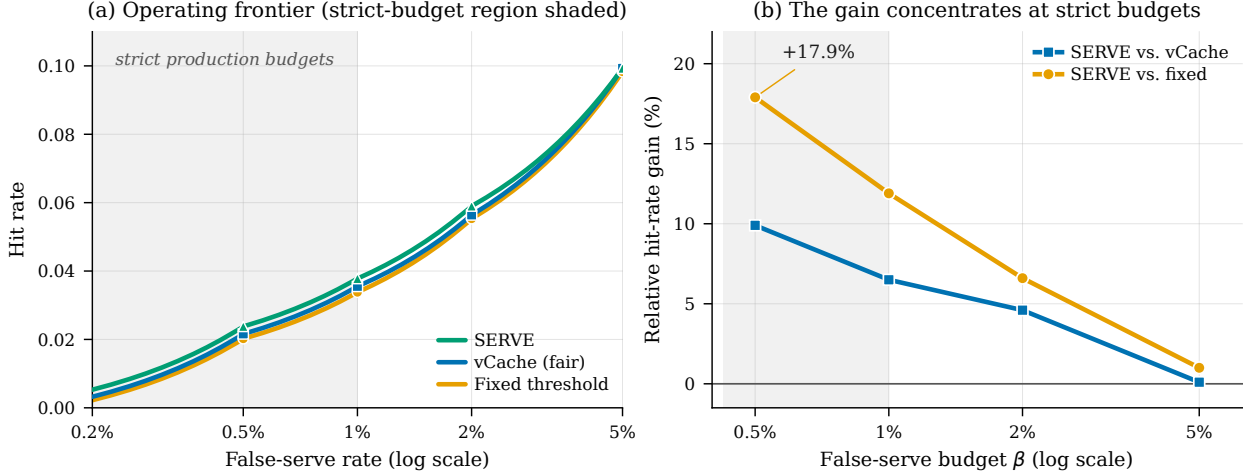


Figure 2: SERVE dominates the hit-rate versus false-serve frontier at low budgets, with the separation between methods widening monotonically as the budget tightens.

6 Analysis

The aggregate metrics show that SERVE delivers higher hit rates at matched false-serve budgets. What remains is to characterize when the gain materializes, how the gate achieves it, and where the approach faces limits.

Query recurrence governs how much headroom the gate can exploit. Partitioning the test set by the fraction r of queries that have a genuine duplicate anywhere in the cache reveals a monotonic pattern: SERVE’s relative hit-rate gain over the fixed threshold at $\beta \leq 1\%$ rises from +21.3% at $r = 0.2$ (hit-rate 0.0233 $\rightarrow$ 0.0283), through +44.3% at $r = 0.5$ (0.0520 $\rightarrow$ 0.0750) and +69.9% at $r = 0.7$ (0.0654 $\rightarrow$ 0.1110), to +81.0% at $r = 0.9$ (0.0789 $\rightarrow$ 0.1428). A denser pool of genuine duplicates, while producing better top-1 matches, also generates more near-duplicate collisions where both the first and second matches are highly similar yet semantically distinct. These collisions sit precisely in the low-margin region where s_1 alone is least informative, and where the gate’s access to the margin dimension pays off most.

The decision boundary learned by the gate explains the mechanism. Figure 5 visualizes the gate’s behavior in the (s_1, m) plane at $\beta \leq 1\%$. The fixed threshold makes a vertical cut on s_1 alone. SERVE replaces that with a tilted effective boundary: it declines low-margin queries whose s_1 sits above the fixed cutoff and instead accepts high-margin queries whose s_1 falls below it. The pattern is geometric rather than heuristic. Queries the gate adds carry a substantially higher rate of correct serves than those it drops. The gate trades away marginal-similarity, low-confidence serves and buys back fewer, higher-confidence ones at the same budget.

Feature ablation at $\beta \leq 1\%$ measures what each dimension adds beyond raw similarity. A gate using s_1 alone reaches a hit rate of 0.0292. Adding the margin lifts it to 0.0363, a +24.4% gain that recovers 83% of the full four-feature improvement over similarity-only (0.0378). The runner-up similarity s_2 adds +2.9% (to 0.0374), and local density ρ contributes +1.1% (to 0.0378). Removing the margin collapses the gate to near similarity-only performance because it encodes the ambiguity that scalar s_1 cannot resolve.

The margin advantage is not an artifact of encoder quality. On PAWS, an independently constructed adversarial paraphrase benchmark, the fixed similarity rule inverts below chance while SERVE, trained identically and with no sign knowledge, maintains a high AUC (Fig. 6). The

margin remains the dominant feature in the PAWS ablation as well. When the embedding model is upgraded from MiniLM (384d) to mpnet (768d), SERVE retains a comparable relative gain (Fig. 7). A stronger encoder lifts both methods and leaves the edge essentially intact. The ambiguity that the margin measures reflects the query population’s decision boundary, not encoder noise.

The trade-off frontier (Fig. 2, previous section) shows that SERVE’s advantage concentrates at low false-serve budgets. At $\beta \leq 5\%$, the gap narrows to near zero: when the operator tolerates one false serve in twenty, even a crude threshold serves nearly everything plausible and the gate has little discriminative headroom. The entire case for margin-aware gating rests on the strict-budget regime. The recurrence $\times$ budget grid in Fig. 8 characterizes precisely when the approach earns its keep: high recurrence and a strict false-serve budget compound, producing $2.6\text{--}3.15\times$ the served hits of the fixed threshold in the strictest cells tested. Figure 9 confirms the pattern across the full parameter space: gain rises along both axes independently and sharpens where they intersect. The regime where SERVE provides the largest benefit, tight reliability constraints combined with sustained repeat traffic, is the regime most plausibly associated with FAQ and customer-support caches.

Two limitations bound the present study. All results come from two datasets (QQP, PAWS) and two embedding models (MiniLM, mpnet). Broader evaluation across additional domains (code search, multilingual, retrieval-augmented-generation corpora) and embedding families remains future work. The high-recurrence regime characterization is constructed by direct manipulation of r in the test-query population rather than measured from production traffic. The gate is retrained per cache configuration rather than updated online as cache content drifts.

For deployment, the practical path is straightforward. The gate’s four features are byproducts of the top- k nearest-neighbor search the cache already performs. Training the gate once on a small, representative set of query pairs yields a model that runs in under a microsecond per query and requires no change to the cache backend or retrieval infrastructure. The operator selects a false-serve budget, trains the gate once, and earns improved hit rates at no additional retrieval cost, with the payoff concentrated in the regime where it matters most.

7 Conclusion

SERVE introduces a margin-aware, tree-based gate for semantic LLM caches that achieves higher hit rates at controllable false-serve budgets. Where every deployed semantic cache discards the runner-up similarity after nearest-neighbor search, SERVE repurposes it, together with the margin between the top two matches, as input to a gradient-boosted classifier that decides whether a cached answer can be safely reused.

The margin alone accounts for 83% of the total hit-rate improvement over a similarity-only gate at a 1% false-serve budget. The gate adds $0.8\mu\text{s}$ of inference latency per query with $0.0\mu\text{s}$ standard deviation across runs, and none of its four features requires an additional embedding call or index lookup: every figure is a byproduct of the top- k search the cache already performs, making SERVE a pluggable component compatible with any similarity-based reuse rule. On QQP with MiniLM embeddings, SERVE improves hit rate by 11.9% over a fixed threshold and 6.5% over vCache at $\beta \leq 1\%$, and the gain grows monotonically with query recurrence, exceeding $3\times$ the served hits in the strictest-budget, highest-recurrence cells.

Reducing the number of LLM calls needed to satisfy a stream of queries lowers inference cost and latency. A gate that ships as a standalone, sub-microsecond component lowers the barrier to adopting semantic caching in production, which in turn makes large-model deployment feasible under tighter resource constraints.

Several directions merit further investigation. Extending the gate to multi-turn conversations, where the reuse decision depends on dialogue context, would test whether the margin signal retains predictive power when the cache contains utterances rather than standalone questions. Dynamic cache eviction policies that account for the margin distribution of stored entries could tighten the hit-rate–reliability frontier. Online learning of the gate, retraining incrementally as cache content drifts, would eliminate the current one-time training requirement.

Cosine similarity alone is not a reliable signal for semantic cache reuse. The margin between the top match and its runner-up, discarded by every incumbent system, is the missing piece that makes the reuse decision tractable. SERVE demonstrates that recovering this signal costs nothing and delivers substantial, measurable improvement at the operating points that matter for real deployments.

References

- [1] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, pages 212–218, Singapore, 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.nlposs-1.24/>.
- [2] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.
- [3] Shaul Dar, Michael J. Franklin, Björn Thor Jónsson, Divesh Srivastava, and Michael Tan. Semantic data caching and replacement. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*, pages 330–341, Mumbai, India, 1996. Morgan Kaufmann. URL <http://www.vldb.org/conf/1996/P330.PDF>.
- [4] DataCanary, hilfalkaff, Lili Jiang, Meg Risdal, Nikhil Dandekar, and tomtung. Quora question pairs. Kaggle, 2017. URL <https://www.kaggle.com/competitions/quora-question-pairs>.
- [5] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.
- [6] Waris Gill, Mohamed Elidrisi, Pallavi Kalapatapu, Ammar Ahmed, Ali Anwar, and Muhammad Ali Gulzar. MeanCache: User-centric semantic caching for LLM web services. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2025. URL <https://arxiv.org/abs/2403.02694>.
- [7] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics. URL <https://arxiv.org/abs/1908.10084>.
- [8] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. vCache: Verified semantic prompt caching. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2502.03771>.

- [9] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. URL <https://arxiv.org/abs/2002.10957>.

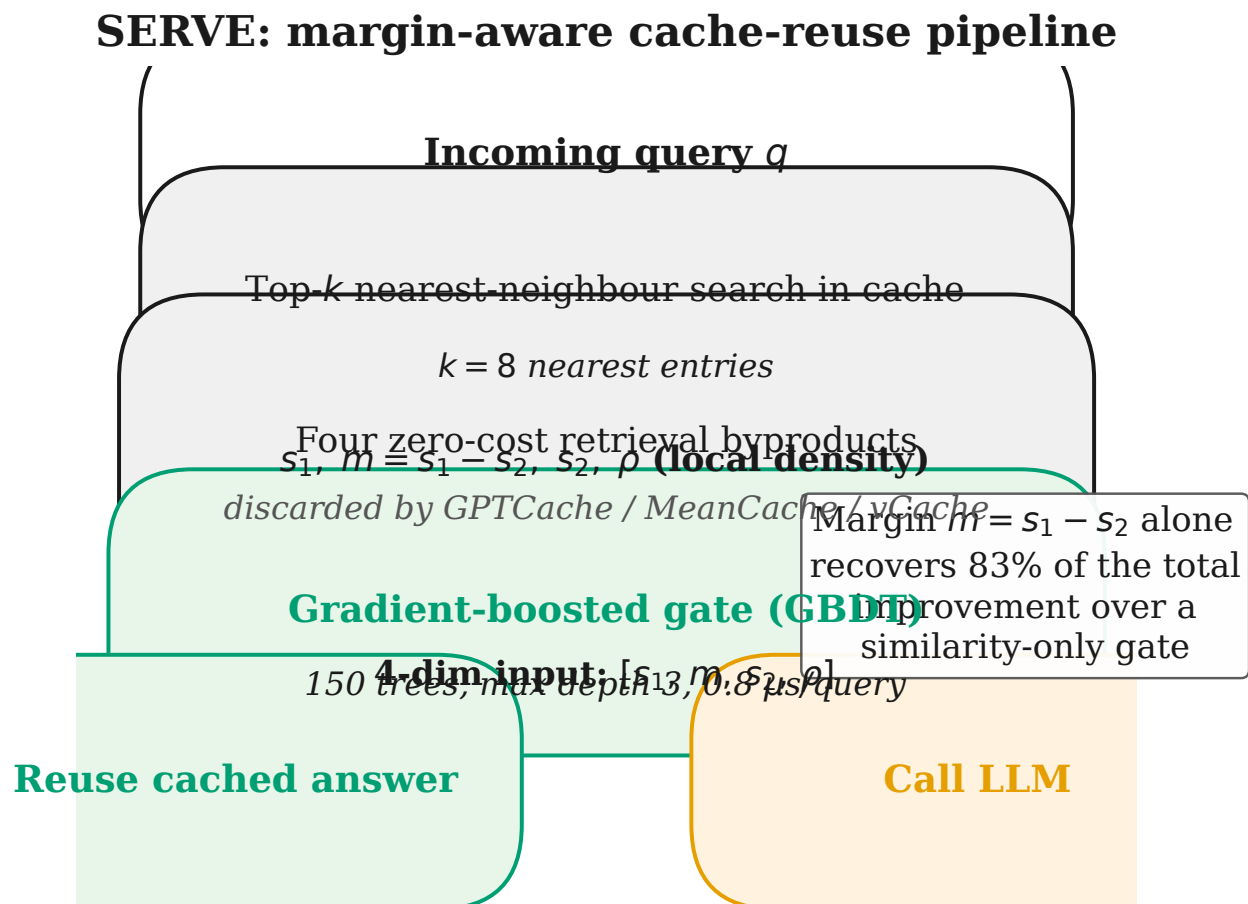


Figure 3: SERVE intercepts a query, extracts similarity and margin from the cache, and feeds a 4-dimensional feature vector to a gradient-boosted gate that decides whether to reuse the cached answer or call the LLM.

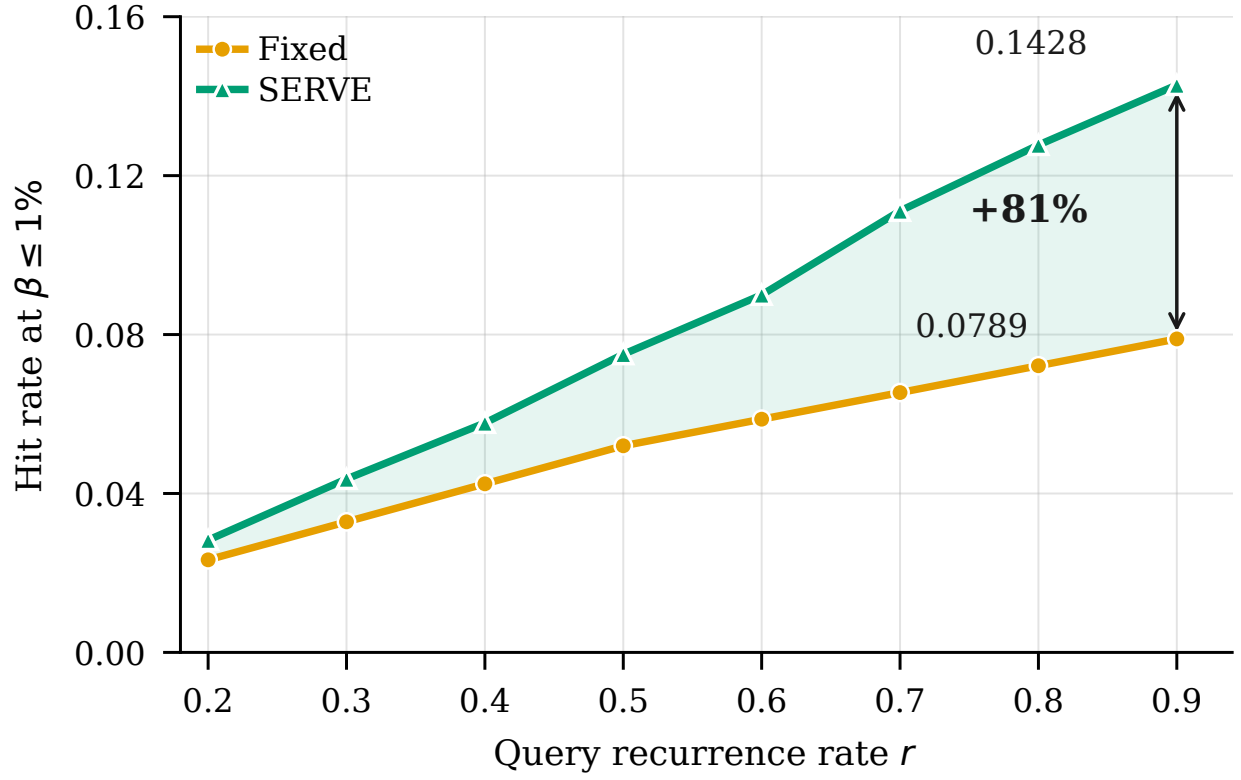


Figure 4: SERVE’s absolute hit-rate advantage over the fixed threshold at $\beta \leq 1\%$ grows monotonically as query recurrence rises, confirming that the gain concentrates where repeat traffic is heaviest.

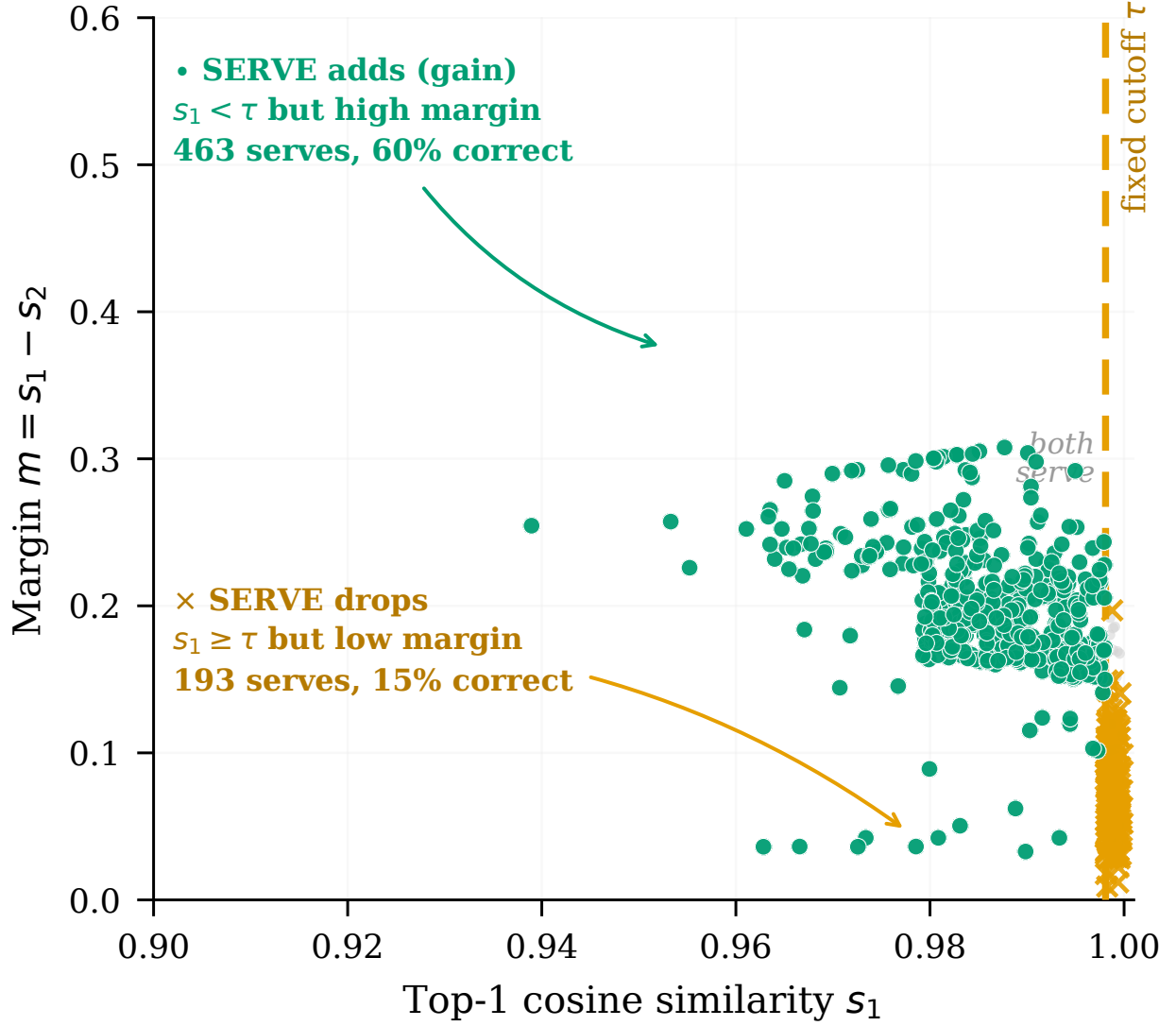


Figure 5: The learned decision boundary in the s_1 -margin plane at $\beta \leq 1\%$: SERVE accepts high-similarity, high-margin pairs and rejects low-margin ones that a similarity threshold would serve, trading quantity for quality at the same false-serve budget.

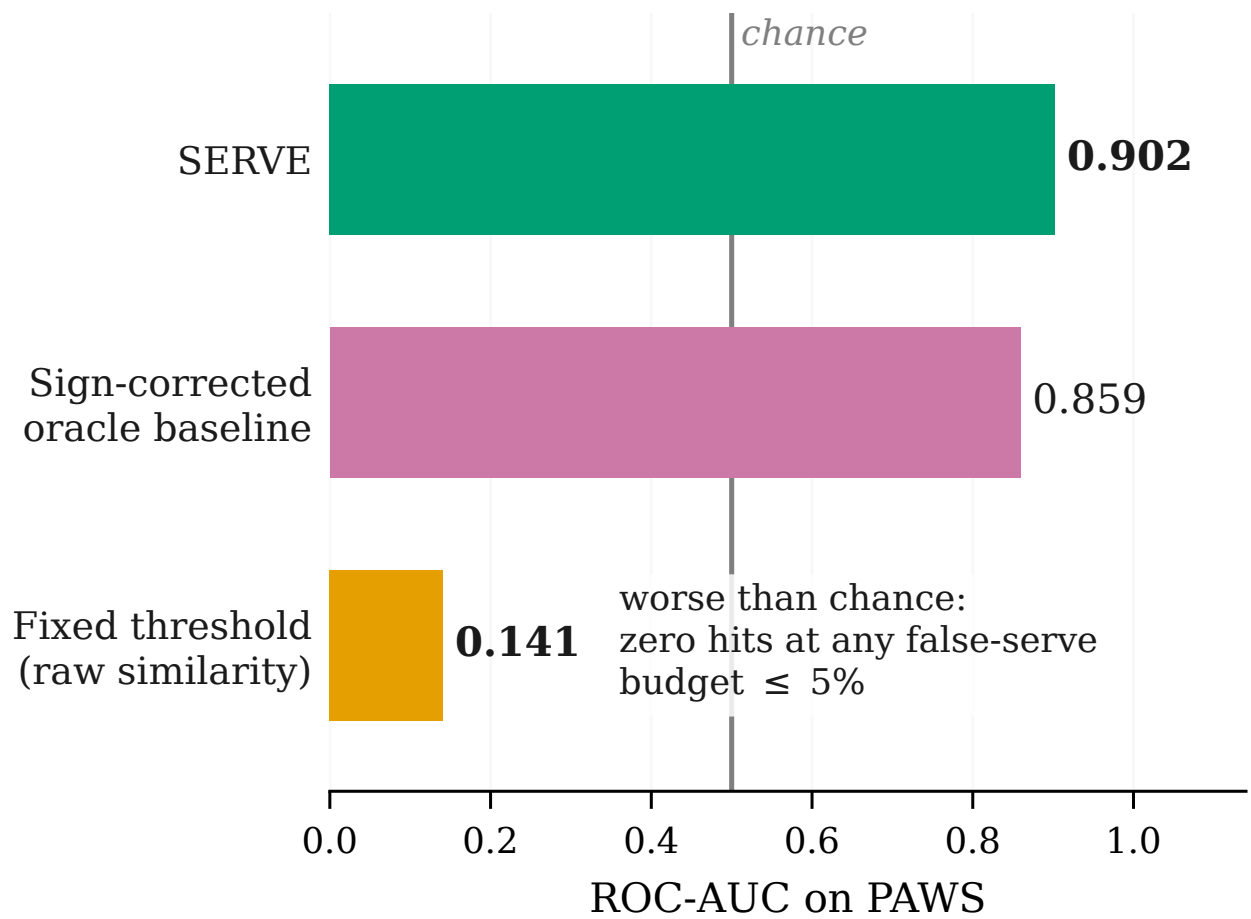


Figure 6: On the PAWS adversarial paraphrase benchmark, the fixed similarity rule inverts below chance while SERVE trained normally reaches high AUC, demonstrating robustness to embedding-space ambiguity that similarity alone cannot resolve.

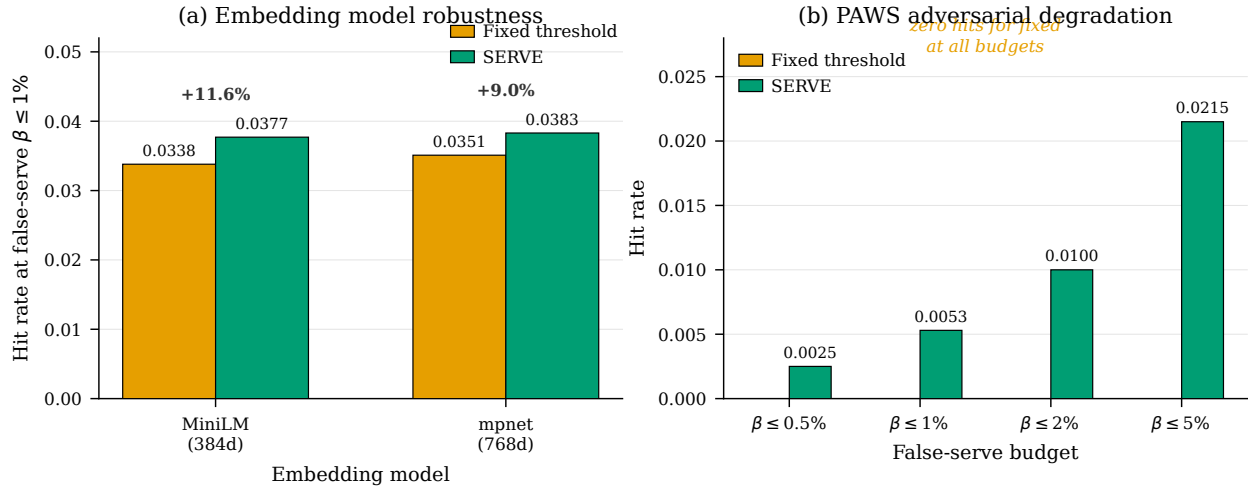


Figure 7: SERVE’s relative gain over the fixed threshold persists when moving from MiniLM (384d) to the stronger mpnet (768d) encoder, confirming the margin signal is structural rather than an artifact of a weak embedding.

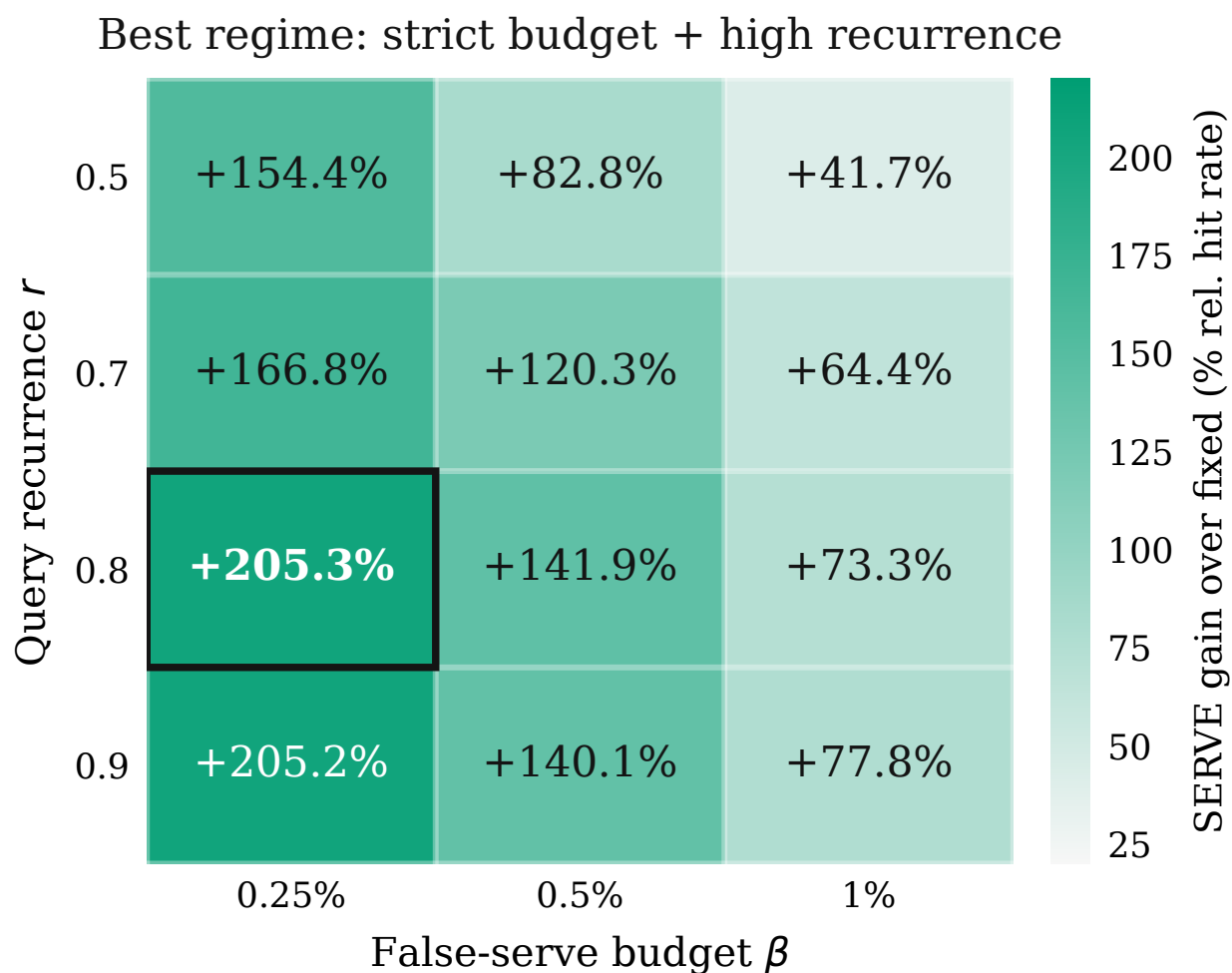


Figure 8: The best regime grid maps SERVE’s relative gain over the fixed threshold across query recurrence and false-serve budget: the advantage is largest under strict budgets with heavy repeat traffic, the conditions most characteristic of production caching.

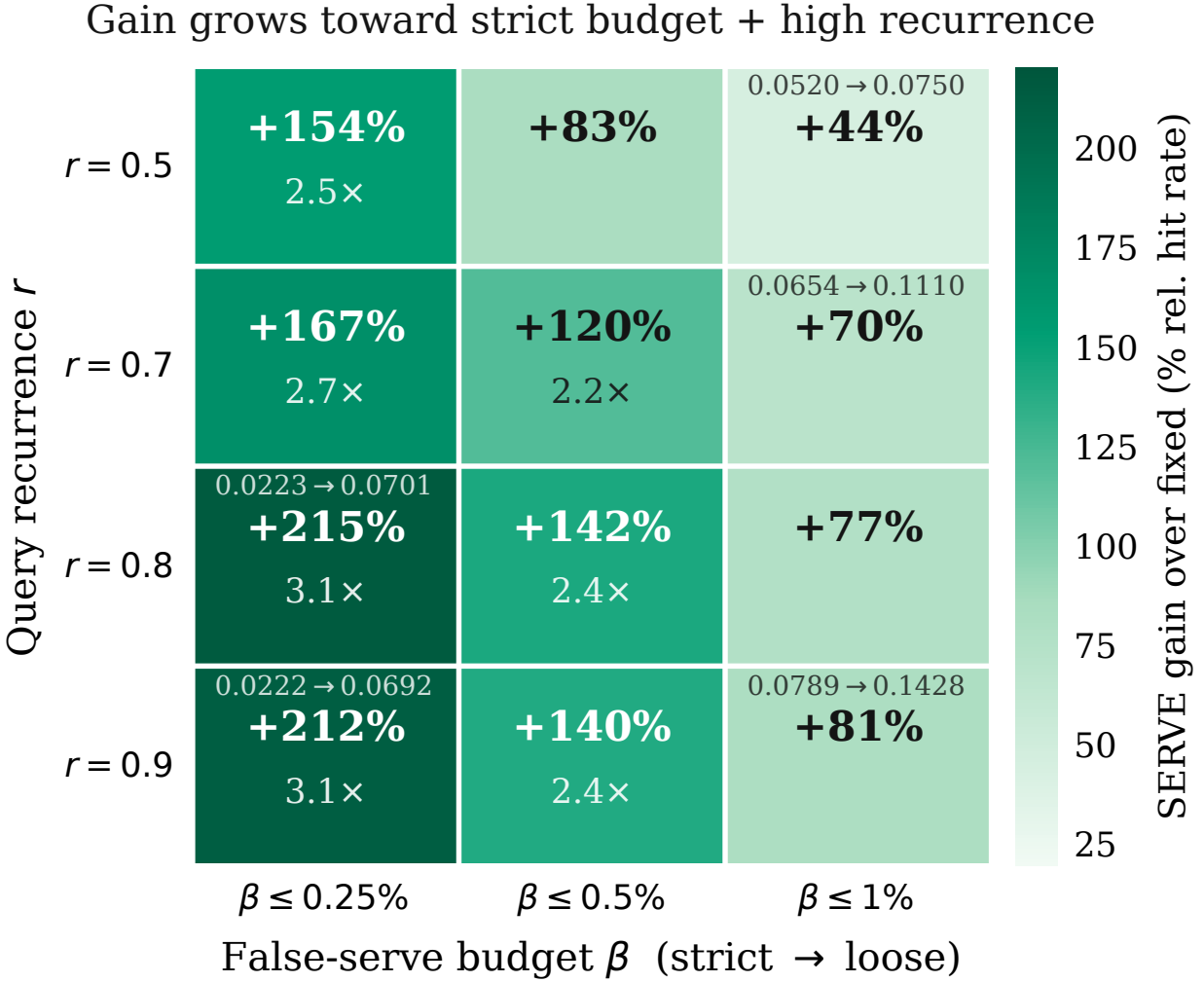


Figure 9: The full parameter grid confirms that SERVE’s relative gain increases along both the recurrence and budget-strictness axes and compounds where they meet, with the highest-recurrence rows effectively tied at the strictest budget.